CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY
Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)

Practical File – ODD Semester 2026–27

Student ID: 24DIT074

Student Name: NANDAN VAKANI

PRACTICAL 1

1. Practical Definition: Understanding the Data Engineering Lifecycle

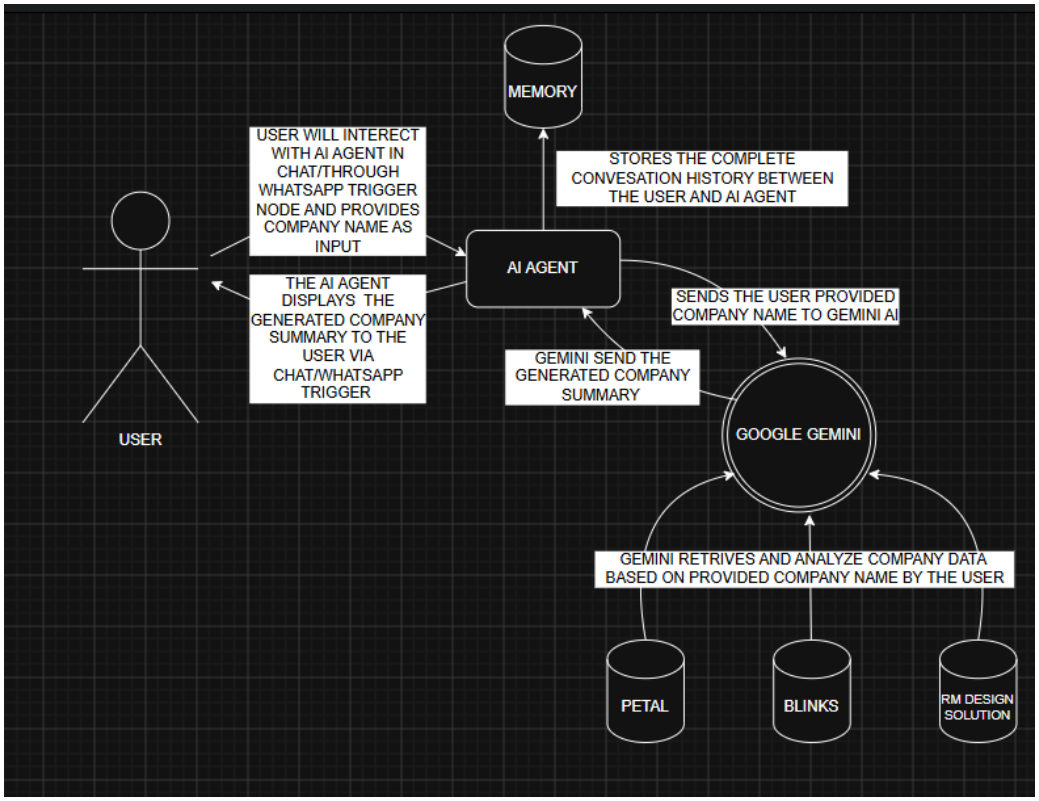
Study, dissect, and map the complete end-to-end data engineering lifecycle for an enterprise e-commerce platform. Students must trace data states across all five core lifecycle stages: data generation, ingestion, storage, transformation, and serving. The solution requires creating a detailed architectural data flow diagram along with a system manifesto documenting cross-cutting lifecycle undercurrents (security, observability, and data privacy). This initial structure must be committed to a personal Git repository.

Dataset/Test Data: Mock enterprise e-commerce dataset schema definition (containing transactional database entities, clickstream tracking JSON payloads, and third-party inventory log streams).

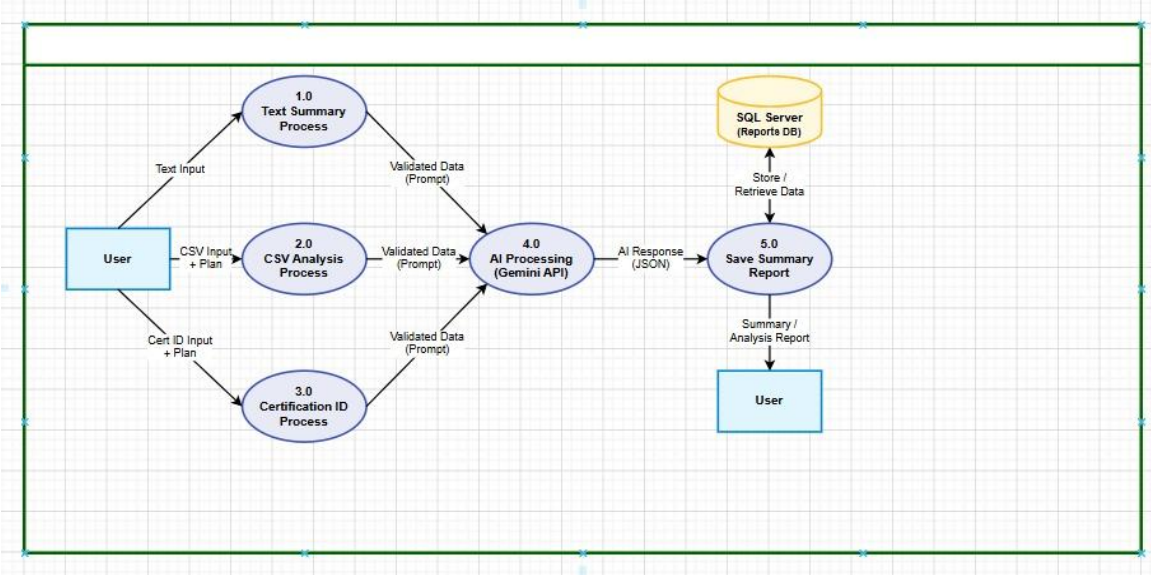
Tools/Technology To Be Used: Draw.io, Lucidchart, Git, and GitHub/GitLab.

Solution : (INTERNSHIP PROJECT : AI SUMMARY GENERATOR)

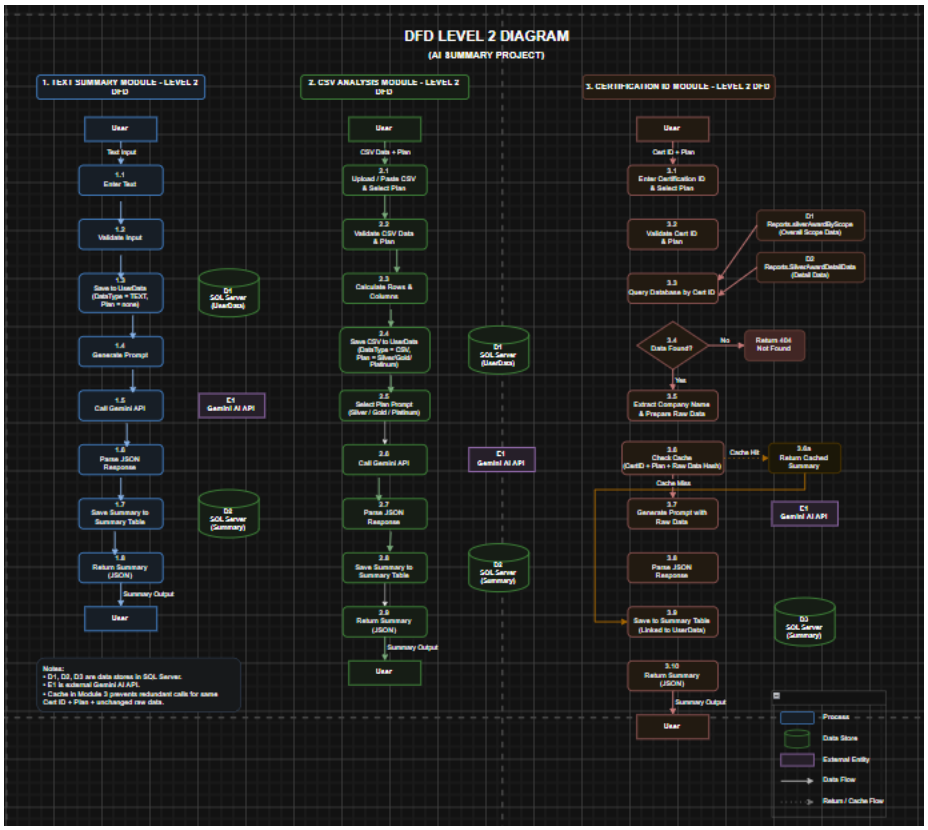
1. DFD Diagram : Level 0



2. DFD Diagram : Level 1



3. DFD Diagram : Level 2



● Question / Key Point 1: How do the technical demands of downstream serving layers directly influence ingestion and storage design decisions during the early generation phases?

Answer : The serving layer is the final consumer of data, and its requirements ripple backwards through every earlier stage of the lifecycle. If the serving layer needs real-time dashboards updated every few seconds, the ingestion layer cannot use nightly batch jobs — it must implement continuous streaming ingestion. If analysts need to query five years of sales data broken down by region and product category, the storage layer must partition data using those exact dimensions (region, product category, date) to avoid slow full-table scans. If machine learning models at the serving layer require enriched customer profiles combining clickstream behavior with purchase history, the transformation layer must produce those joins, and the ingestion layer must ensure both data streams arrive with aligned timestamps. In short, you cannot design ingestion and storage in isolation — you must first understand what the serving layer demands in terms of latency, query patterns, data freshness, and data structure, and then design every upstream stage to satisfy those requirements.

● Question / Key Point 2: Differentiate between the broader data lifecycle and the technical data engineering lifecycle within an enterprise.

Answer : The broader data lifecycle refers to the complete journey of data from the moment it is created as a business event all the way through its use, archiving, and eventual deletion. It includes business-level stages such as data creation, data capture, data processing, data analysis, data publishing, and data archiving or disposal. This lifecycle is governed by organizational policies, legal compliance requirements, and business objectives. It is concerned with questions like: who owns this data, how long must we keep it, and what are we legally permitted to do with it?

The technical data engineering lifecycle is a subset of this broader lifecycle. It focuses specifically on the engineering stages that a data engineer is responsible for building and maintaining: data generation (interfacing with source systems), ingestion (moving data reliably into the platform), storage (organizing and persisting data efficiently), transformation (cleaning and reshaping data for analysis), and serving (delivering data to consumers). While the broader lifecycle includes governance, legal, and business concerns, the technical data engineering lifecycle is concerned with the systems, pipelines, tools, and architectural patterns that make data reliably available for analytical and operational use.

● Question / Key Point 3: Identify how the undercurrents of security and data observability should be enforced at each lifecycle boundary.

Answer : Security and observability are not features added at the end of a pipeline — they are undercurrents that must be built into every boundary between lifecycle stages.

At the boundary between **generation and ingestion**, security requires that source systems authenticate themselves before pushing data, and that all data in transit is encrypted. Observability requires logging the volume and frequency of incoming data so that missing or delayed feeds are detected immediately.

At the boundary between **ingestion and storage**, security requires that only authorized pipeline services have write access to the raw storage zone, and that PII fields are tagged or masked before landing. Observability requires that every ingestion job logs its completion status, record counts, and any schema anomalies encountered.

At the boundary between **storage and transformation**, security requires that transformation jobs run under least-privilege service accounts and that access to raw storage is audited. Observability requires monitoring data freshness — if transformation jobs consume yesterday's data when they expected today's, an alert must fire.

At the boundary between **transformation and serving**, security requires that only clean, masked data reaches the serving layer and that role-based access controls are applied to all views and data marts. Observability requires tracking query performance, data staleness at the serving layer, and any unauthorized access attempts.

Enforcing these undercurrents at every boundary — rather than only at the entry or exit point of the pipeline — ensures that a failure or breach at any single stage does not compromise the integrity or security of the entire system.